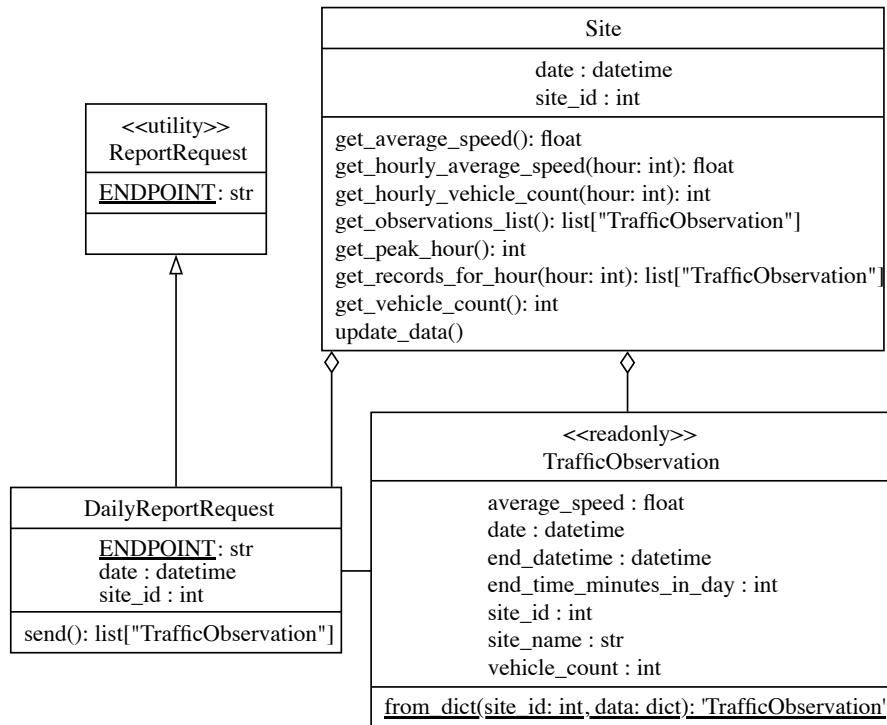


Fundamentals of CS II AE2

This report provides in-depth explanations and justifications for the decisions made while designing and implementing the code in this project, AE2. UML-style diagrams based on UML 2.5.1 are included to demonstrate class structure visually.

Class Layout



The UML diagram models the class layout for the four classes in the program. `ReportRequest` is an abstract utility class and is not meant to be instantiated. `DailyReportRequest` inherits from `ReportRequest`. `Site` aggregates `DailyReportRequest` and `TrafficObservation` objects.

Class Descriptions

ReportRequest

This class is not meant to be instantiated. It only contains the static `ENDPOINT` attribute for report requests. `ReportRequest` is intended to be inherited so future implementations can generalize report requests, since "monthly" and "annual" options are not yet implemented and differ from the current "daily" option. Potential usage as a parent class:

```
class DailyReportRequest(ReportRequest):
    ...
```

Potential usage to access `ENDPOINT`

```
URL = f"{API_URL}{ReportRequest.ENDPOINT}"
```

DailyReportRequest

The `DailyReportRequest` class is used to request daily traffic-camera data in fifteen-minute intervals from the endpoint. Users may instantiate this class directly, but using the `Site` class is recommended because it provides helper methods built around this request class. `DailyReportRequest` takes a `site_id` and `date` on initialization so it can query data relevant to that date and site. Its main functionality is contained in the `.send()` method. If any returned data is faulty (excluding unnecessary fields), it is excluded from the returned list. Even if one traffic observation lacks required fields, other observations may still be valid and should be returned. Errors, however, should not stop the application entirely.

```
req = DailyReportRequest(site_id, date)
observations: list[TrafficObservation] = req.send()
```

TrafficObservation

`TrafficObservation` is primarily returned by `DailyReportRequest.send()` and is not meant to be instantiated by the user. During initialization, if fields are invalid, `TrafficObservation` refuses to store faulty or empty values for `site_name`, `site_id`, `observation_end_datetime`, `average_speed`, and `vehicle_count`, instead raising a `TypeError` or `ValueError`. The rationale for this implementation is that observations are intended to be used for comparison and analysis. Calculating an average speed for the day, for example, requires vehicle counts to weight each fifteen-minute average speed correctly.

Example usage:

```
req = DailyReportRequest(site_id, date)
observations = req.send()
for o in observations:
    print(f"Avg speed for time {o.end_datetime.strftime('%H:%M:%S')}: {o.average_speed}")
```

Site

A class representing one site's observations for a given date. It takes `date` and `site_id` arguments and queries the API on initialization and whenever the date is updated. `Site` also refreshes its internal data when `date` is updated to guarantee that its data is always in sync with its attributes (`site_id` is read-only to encourage the pattern of a `Site` representing only one site). This class is the primary outward-facing class intended for users. It provides many helper methods for operating on its internal list of `TrafficObservation` objects. Data can be queried by multiple methods in the class to draw conclusions from the observations. Example usage:

```
site = Site(site_id=site_id, date=date)
print(f"Site Name: {site.name or 'Unknown Name'}")
print(f"Average Speed: {site.get_average_speed()} mph")
print(f"Vehicle Count: {site.get_vehicle_count()}")
```

Alternatives and Trade-offs

Immutability

In `TrafficObservation`, all data fields are either protected or immutable. They were designed this way because the API should be the exclusive, authoritative source of information. Allowing users to modify attributes on `TrafficObservation` could create situations where modified data is later treated as authoritative API data. The best way to maintain data integrity is to keep these fields immutable.

Type Checking

To ensure that data is valid during initialization, the class performs multiple type and value checks on input data to ensure values are both type-compliant and contextually valid for the project. Average speed, for example, should never be less than zero. If any imposed condition is not met, a `ValueError` or `TypeError` is raised. These errors can either be caught and the affected data ignored, or allowed to propagate and end the program, with the former being recommended.

Data Exclusion

In `DailyReportRequest`, if a full `TrafficObservation` cannot be formed due to partially empty or faulty API data, it is excluded without raising an error. Using `None`, averages, or zeroes to fill missing data was considered, but predicted use cases for this codebase (such as averaging data over an entire day) typically rely on multiple fields from each observation. Data containing many `None` values would require repeated `is None` checks, whereas excluding invalid observations before returning results allows downstream implementations to stay cleaner and rely on data integrity.

If a site has no valid observations, `Site` methods return `0`. If it is necessary to differentiate this from a genuinely valid day with zero traffic, `len(site) == 0` indicates that there are no valid observations.

Example Interactions

Example User-Written Program

Below (`main.py`) is an example user program that asks for a site ID and date, then prints traffic data for that site.

```

"""Demonstration on how to use the application"""

from datetime import datetime
from webtris_client import Site

if __name__ == "__main__":
    site_id = input("Enter a site ID: ")
    try:
        site_id = int(site_id)
    except ValueError:
        print("Invalid site ID. Please enter a valid integer.")
        exit(1)
    date = input("Enter a date (YYYY-MM-DD): ")
    try:
        date = datetime.strptime(date, "%Y-%m-%d")
    except ValueError:
        print("Invalid date format. Please enter the date in YYYY-MM-DD
format.")
        exit(1)

    # Get the traffic data for the site and date
    site = Site(site_id=site_id, date=date)
    print(f"Site Name: {site.name or 'Unknown Name'}")
    print(f"Average Speed: {site.get_average_speed()} mph")
    print(f"Vehicle Count: {site.get_vehicle_count()}")
    peak_hour = site.get_peak_hour()
    print(f"Peak Hour: {peak_hour}
({site.get_hourly_vehicle_count(peak_hour)} vehicles)")

    """
    Example usage:
    Enter a site ID: 14
    Enter a date (YYYY-MM-DD): 2025-03-10

    Site Name: A2/8392M
    Average Speed: 42.467943045611776 mph
    Vehicle Count: 12431
    Peak Hour: 8 (1094 vehicles)
    """

```

Example Process Diagram

Below is an example of the process which happens when a user initializes a `Site` and calls `get_observations_list()`.

User Interactions

